

FoldShield++

UI Setup Guide for Front-End Development

RexCrux Research Laboratory — Confidential

1. Purpose of This Document

This guide covers everything needed to run the FoldShield++ Streamlit UI locally, load sample data, and explore the interface — without requiring access to the core pipeline code.

Scope of this package: You are receiving `app.py` (the full UI script), a set of sample PDB structure files, and pre-computed `summary.json` result files. The pipeline engine and all core modules are not included — the UI can operate in a read-only demo mode by loading the provided summary files directly.

2. What You Are Working With

FoldShield++ is a protein structural similarity tool. The interface has three tabs:

- **Visuals** — 3D backbone visualisation, signal bar chart, radar chart.
- **Results** — Numerical scores for each similarity signal, combined score, perturbation classification, and a PDF download button.
- **Log** — Raw JSON output and run log (pipeline mode only).

The sidebar has two operating modes:

- **Live pipeline mode:** Upload two PDB files and click Run pipeline. Requires the full codebase — not available with this package.
- **Demo / JSON mode:** Upload a `summary.json` file directly. Results and visuals load instantly. This is the primary mode for front-end work.

3. Files in This Package

File / Folder	Description
<code>app.py</code>	Main Streamlit UI script — the only source file shared
<code>samples/pdbs/</code>	Sample PDB structure files for visual testing
<code>samples/results/</code>	Pre-computed <code>summary.json</code> files, one per benchmark pair

samples/results/*/entropy_ref.png	Entropy heatmap images referenced by summary.json
samples/results/*/entropy_query.png	Entropy heatmap images referenced by summary.json

Note: The `summary.json` files point to image paths. Keep the folder structure intact — `entropy_ref.png` and `entropy_query.png` must sit alongside `summary.json` in the same directory.

4. Environment Setup

4.1 Prerequisites

Requirement	Version / Notes
Python	3.10 or 3.11 recommended
pip	Latest (comes with Python)
Git (optional)	Only if cloning; not required if files are zipped

4.2 Create a virtual environment

Open a terminal in the folder containing `app.py` and run:

```
# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate

# Windows (Command Prompt)
python -m venv .venv
.venv\Scripts\activate
```

4.3 Install dependencies

The UI depends on a small set of packages. Install them with:

```
pip install streamlit plotly numpy
```

That is the complete list required for demo / JSON mode. No BioPython, no GUDHI, no pipeline packages are needed.

4.4 Stub files required by `app.py`

app.py imports several modules from the core pipeline. These do not need to do anything in demo mode, but Python must be able to import them without crashing. Create the following stub structure in the same directory as app.py:

```
mkdir -p core/pdb core/similarity visualisation
touch core/__init__.py
touch core/pdb/__init__.py
touch core/similarity/__init__.py
touch visualisation/__init__.py
```

Then create these four stub files with the content shown below.

core/pdb/pdb_precheck.py

```
def analyze_pdb(path):
    return {'chain_ids': ['A']}
```

core/similarity/fusion_layer.py

```
import yaml, os

class SignalNormalizer:
    @classmethod
    def from_yaml(cls, path):
        return cls()
    def transform(self, x):
        return x

class RidgeFusionLayer:
    def __init__(self, weights_yaml=None, mode='phase2'):
        pass
```

visualisation/visualise_feature_extractor.py

```
import plotly.graph_objects as go

def plot_dual_pdb_token_backbones_plotly_separate(
    ca_ref, tokens_ref, ca_query, tokens_query):
    fig = go.Figure()
    fig.update_layout(title='3D Backbone (stub - no core module)')
    return fig
```

configs/weights_phase2.yaml

app.py looks for a YAML config at startup. Create the configs folder and an empty YAML:

```
mkdir configs
```

```
# contents of configs/weights_phase2.yaml
phase2:
  braid:      0.20
  motif:     0.15
  topology:   0.20
  local_topo: 0.25
  ssef:      0.20
```

5. Running the App

With the virtual environment active and the stubs in place, start Streamlit:

```
streamlit run app.py
```

Streamlit will print a local URL — typically `http://localhost:8501`. Open it in a browser.

6. Loading Sample Results (Demo Mode)

1. In the left sidebar, locate the Upload summary.json field (below the PDB uploaders).
2. Click Browse files and select one of the summary.json files from samples/results/.
3. The Results and Visuals tabs will populate immediately — no pipeline run needed.

Each summary.json corresponds to one benchmark protein pair. The file name indicates which pair it covers (e.g. ubiquitin_1ubq_1ubi.json, adenylate_kinase.json).

Note: *If the entropy images do not appear in the Visuals tab, check that entropy_ref.png and entropy_query.png are in the same folder as the summary.json you loaded, and that the outdir path inside the JSON matches that folder.*

7. Summary JSON — Key Fields

The following table describes the fields app.py reads from summary.json. This is useful for understanding what drives each UI element.

JSON key	UI element it drives
tm_score, rmsd	Coordinate baseline cards (Results tab)
fusion.signal_scores	Signal score cards and bar chart / radar chart
fusion.weights_used	Weight displayed under each signal card
fusion.per_signal	Interpretation text in the signal table
fusion.combined_score	Large combined score display

fusion.overall	Verdict text (e.g. High similarity)
fusion.flags	Warning banners below the verdict
phase2.ssef_score	SSEF score tile (Phase 2 section)
phase2.local_topology_score	Local topology tile
phase2.gap_score	Gap score tile and colour coding
phase2.perturbation_class	Perturbation class label
ca_coords_ref, ca_coords_query	3D backbone trace data
tokens_ref, tokens_query	UL-DSL token colours on backbone
inputs.outdir	Path used to locate entropy PNG images

8. Complete Folder Structure

After setup, the working directory should look like this:

```
foldshield-ui/
├── app.py
├── configs/
│   └── weights_phase2.yaml
├── core/
│   ├── __init__.py
│   ├── pdb/
│   │   ├── __init__.py
│   │   └── pdb_precheck.py
│   └── similarity/
│       ├── __init__.py
│       └── fusion_layer.py
├── visualisation/
│   ├── __init__.py
│   └── visualise_feature_extractor.py
└── samples/
    ├── pdbs/
    │   ├── 1ubq.pdb
    │   └── ...
    └── results/
        ├── ubiquitin/
        │   ├── summary.json
        │   ├── entropy_ref.png
        │   └── entropy_query.png
        └── ...
```

9. Design Scope — What Can Be Changed

All visual design decisions live inside `app.py`. The following elements are straightforward to restyle:

Element	Where to find it in <code>app.py</code>
Colour palette (red / amber / green thresholds)	<code>_color()</code> function, ~line 227
Score card layout and typography	Inline HTML in the Results tab section
Gauge charts (combined score)	<code>gauge()</code> function, ~line 66
Progress bar meters	<code>meter()</code> function, ~line 37
Bar chart colours and layout	<code>go.Bar()</code> call in the Visuals tab
Radar chart fill and line style	<code>go.Scatterpolar()</code> call in the Visuals tab
Tab labels	<code>st.tabs()</code> call, ~line 94
Page title and header	<code>st.set_page_config()</code> and <code>st.title()</code> , lines 10–11

The core data flow (sidebar → session state → tab rendering) should not be restructured without checking with the engineering team, as it depends on Streamlit's execution model.

10. Troubleshooting

Problem	Fix
<code>ModuleNotFoundError: core.pdb.pdb_precheck</code>	Create the stub files as described in Section 4.4
<code>ModuleNotFoundError: core.similarity.fusion_layer</code>	Same — stub file missing
<code>FileNotFoundError: configs/weights_phase2.yaml</code>	Create the configs folder and YAML as in Section 4.4
Entropy images not showing in Visuals tab	Check that <code>entropy_ref.png</code> / <code>entropy_query.png</code> are in the same folder as <code>summary.json</code> , and that the <code>outdir</code> value in the JSON points to that folder
Streamlit command not found	Activate the virtual environment first: <code>source .venv/bin/activate</code>
Port 8501 already in use	Run: <code>streamlit run app.py --server.port 8502</code>

11. Questions

For anything not covered here, reach out to the engineering team before modifying the data flow or import structure. Design changes to HTML/CSS blocks and chart styling are fully self-contained and safe to iterate freely.

